An Automated DevOps Framework for Cloud-Native 5G-Advanced Network Deployment and Optimization

23CSE498 Project Phase 2 — Review 1

Team ID: 23UG005 • Panel No. 4 • 17 August 2026

S.No	Reg.No	Name of the Student	Branch
1	BL.EN.U4CSE23208	Anvita Arasavilli	CSE
2	BL.EN.U4CSE23224	Janhavi Nilesh Parate	CSE
3	BL.EN.U4CSE23233	Keerthi Devarajan Anuradha	CSE
4	BL.EN.U4CSE23264	Mukkara Rithvik Reddy	CSE

Project Guide: **Ms. Pooja Gowda**

Department of Computer Science and Engineering

Guide’s Approval

Approval

Screenshot of the guide’s approval mail to proceed with this presentation is placed here.

[Insert screenshot of the approval e-mail here]

replace this box with the image before the review

Problem Definition

Problem

CI/CD pipelines are designed for stateless IT applications. They end at build–test–deploy and ignore runtime network behaviour.

Impact

Fragmented lifecycle management of 5G Core and RAN; latency, throughput and slicing guarantees remain unverified in live networks.

Need

A DevOps framework in which deployment, orchestration and continuous assurance form a single closed loop.

Build → Test → Deploy → Runtime assurance *not covered*

Literature Survey (1/2)

#	Authors	Title (year)	Inference	Open problem
1	Makani & Jangampeta	Evolution of CI/CD Tools in DevOps: Jenkins to GitHub Actions (2022)	Shift to repository-driven workflows	Pipelines unaware of distributed edge and high latency
2	Moravcik et al.	Kubernetes — Evolution of Virtualization (2022)	Containers improve resource efficiency	Auditing heterogeneous deployments
3	Bhatia	Evolution of CI/CD in Cloud-Native Development (2025)	Pipelines moving to event-driven agents	Logic scattered across tools
4	Rigneault et al.	Cloud-Native Network Slice Orchestration in 5G and Beyond (2023)	Cloud-native telecom still maturing	**Resource allocation per slice unresolved**

Literature Survey (2/2)

#	Authors	Title (year)	Inference	Open problem
5	Panek et al.	5G/5G+ Network Management Employing AI-Based Continuous Deployment (2023)	AI improves KPI accuracy	**KPI validation offline, outside the pipeline**
6	Motamary	A Deep Dive into CI/CD Pipelines Tailored for Telecom (2023)	Telecom-specific CI/CD and IaC proposed	Not validated end to end
7	Herrera et al.	A Tutorial on O-RAN Deployment Solutions for 5G (2026)	Simulation to real testbed approaches	**Testing not integrated with CI/CD**
8	Sufyan et al.	From 5G to Beyond 5G: A Comprehensive Survey (2023)	mMIMO and mmWave gains	Hardware limits constrain theoretical gains

Justification for the Proposed Problem

What the literature leaves open

- Deployment is automated; **runtime assurance is not**
- KPI validation happens after deployment, manually
- Slices are treated as configuration, not as measurable resources
- Testing sits outside the pipeline

Why this work is needed

- A deployment that starts is not a deployment that *works*
- Per-slice guarantees cannot be assured without per-slice visibility
- Reproducibility is a prerequisite for any KPI comparison
- Assurance must be continuous, not a one-time test

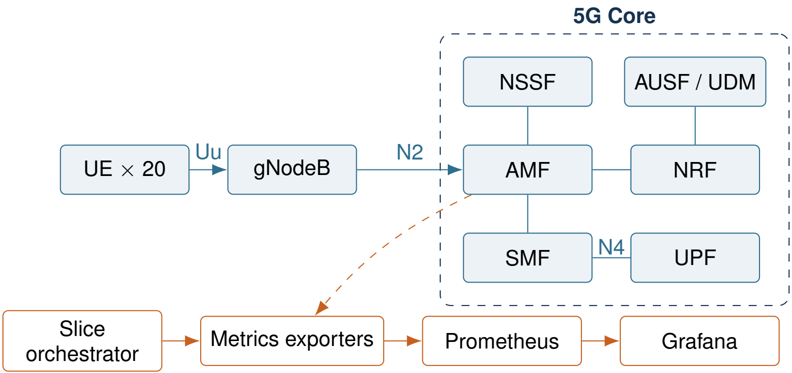
The proposed framework closes the loop between deployment automation and slice-aware runtime observability.

Objectives and Current Status

#	Objective (mapped to the open problems above)	Status
1	Reproducible containerized 5G Standalone Core and RAN	Completed
2	Core–RAN integration and end-to-end UE connectivity	Completed
3	Multiple UEs with differentiated per-UE traffic	Completed
4	Network slicing with distinct service types	Completed
5	Continuous observability plane for runtime assurance	Completed
6	**Slice-aware** metrics for per-slice assurance	Completed
7	Traffic allocation by demand and available capacity	Completed
8	KPI validation as a gate inside the CI/CD pipeline	In progress
9	Migration of the validated baseline to Kubernetes	Planned

Principal challenge: the 5G Core's native metrics carry **no slice identity**, which blocks objective 6 directly — resolved in slide 12.

Architecture of the Proposed System



Design principle

Telemetry flows in **one direction only**. The observability plane reads from the Core and never writes back into it, so monitoring cannot perturb the network under test.

Functionalities of the Proposed System

Deployment and validation

- Reproducible, version-pinned environment
- Automated environment conformance checks
- Integrity gate in continuous integration
- Automated subscriber provisioning

Observability

- Metrics collected from every network function
- Dashboards for runtime network state
- Alert rules for availability and errors
- Timestamped evidence capture

Radio access and mobility

- gNodeB–AMF signalling over SCTP/NGAP
- UE registration with 5G-AKA authentication
- Multiple UEs operating in parallel

Slice management

- Two service types with distinct QoS
- Slice-labelled metrics
- Demand-based allocation with admission control

Software / Tools Requirements

5G network stack

free5GC	5G Standalone Core
UERANSIM	gNodeB and UE simulation
gtp5g	User-plane kernel module
MongoDB	Subscriber database

Platform

Ubuntu 24.04 LTS	Host operating system
Docker + Compose	Containerization
Kubernetes	Orchestration (Phase 2)

Observability and automation

Prometheus	Metrics collection
Grafana	Dashboards
Python	Exporters, orchestrator
Bash	Deployment automation
Git / GitHub Actions	Version control, CI

Hardware baseline

x86_64, $\geq$ 8 logical cores, $\geq$ 16 GB RAM, $\geq$ 100 GB SSD; bare-metal Linux required for the user plane.

Phase 2 Progress — Observability Plane

What was built

- Native metrics enabled on every network function
- Continuous collection every 15 seconds
- Dashboards for registration, signalling and errors
- Alert rules for availability and failure rates

Signals collected

- Registration and session procedures
- NAS and NGAP message rates
- Inter-function API latency
- Error responses by cause

Why it matters

Deployment success is no longer inferred from container status — it is measured from the network's own behaviour.

Phase 2 Progress — Key Finding

Observation

Every metric exposed by the 5G Core was examined. **None of them identifies the network slice it belongs to.**

Consequence

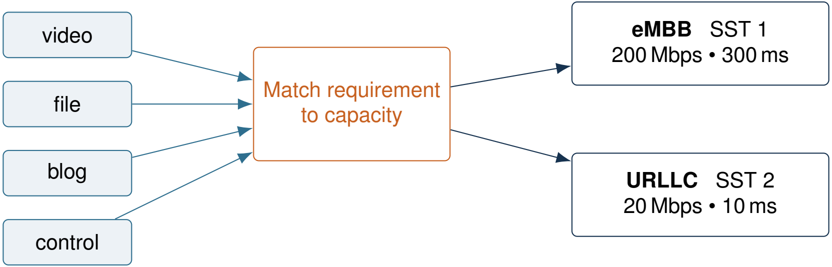
Per-slice assurance is impossible from native telemetry alone. One can see that the network is busy, but not *which slice* is busy.

Resolution — without modifying upstream code

Slice identity is recovered from the Core's own signalling records and joined with the subscriber database to produce genuinely **slice-labelled** metrics.

This confirms an open problem identified in the literature survey: slicing is treated as configuration rather than as an observable, measurable resource.

Phase 2 Progress — Slice-Aware Resource Allocation



Allocation policy

Traffic is placed on the *least specialised* slice that still meets its latency requirement, preserving scarce low-latency capacity.

Admission control

A demand is **refused** when no permitted slice has capacity remaining — resources are finite and enforced.

Results

Deployment

- 8 of 8 network functions registered
- 20 of 20 UEs registered successfully
- Two slices with distinct service types
- Environment reproduces identically

Slice differentiation achieved

Parameter	**eMBB**	**URLLC**
Service type (SST)	1	2
QoS identifier (5QI)	9	82
Priority (ARP)	8	2
Bandwidth ceiling	200 Mbps	20 Mbps
Latency budget	300 ms	10 ms

Verification

All figures are produced by automated checks and stored as timestamped evidence, so every result is reproducible rather than merely reported.

Challenges and Key Learnings

Challenge	Learning / Resolution
Metrics carry no slice identity	Recovered from signalling records instead of patching upstream code
Authentication failures resembling wrong credentials	Root cause was a sequence-number mismatch — symptoms can point away from the real fault
Slice selection failing silently	Configuration must be consistent across *every* network function
Built-in counters drifting from reality	Independent ground truth is essential before trusting a dashboard
User-plane module tied to the host kernel	Environment constraints must be validated before integration, not during

Timeline

08 Aug	30 Aug	20 Sep	05 Oct	20 Oct	30 Oct
Kubernetes orchestrator	CI-CD pipeline; NF checks	Observability plane; slice dashboards	Network slicing via NSSF	KPI en- forcement in pipeline	Document submission

• Completed • Current • Planned

Conclusion

Achieved in this phase

- Reproducible 5G Core and RAN deployment
- Continuous observability plane
- Slice-labelled metrics despite no native support
- Demand-based slice allocation with admission control

Remaining work

- KPI thresholds as pipeline pass/fail gates
- Automatic rollback on KPI regression
- Migration to Kubernetes orchestration
- Allocation driven by measured load

Contribution

The framework moves 5G deployment from *“the containers started”* to *“the network is measurably serving each slice”* — and the remaining work converts that observation into automated action.

References

1. Makani, S. T., & Jangampeta, S. (2022). The Evolution of CI/CD Tools in DevOps from Jenkins to GitHub Actions. *Int. J. of Computer Engineering*.
2. Moravcik, M., Kontsek, M., Segec, P., & Cymbalak, D. (2022). Kubernetes — Evolution of Virtualization. *ICETA*.
3. Bhatia, A. (2025). Evolution of CI/CD in Cloud-Native Development. *IJCTT*, 73(5), 155–161.
4. Baqar, M., Naqvi, S., & Khanda, R. (2025). AI-Augmented CI/CD Pipelines: From Code Commit to Production with Autonomous Decisions. *PriMera Scientific Engineering*.
5. Owobu, W. O., Gbenle, P., & Alozie, C. E. (2025). The Evolution and Impact of Kubernetes in Modern Software Engineering.
6. Rigneault, H., et al. (2023). Cloud Native Network Slice Orchestration in 5G and Beyond.
7. Panek, M., Pomykała, A., Jabłoński, I., & Woźniak, M. (2023). 5G/5G+ Network Management Employing AI-Based Continuous Deployment. *Applied Soft Computing*.
8. Motamary, S. (2023). A Deep Dive into CI/CD Pipelines Tailored for Telecom: Orchestrating Cloud-Native 5G Services with DevOps and Infrastructure Automation.
9. Singirikonda, P. (2024). 5G-Enabled DevOps: Revolutionizing Integrated Communication and Networking Technologies for Faster Software Delivery.
10. Davuluri, S. K., Challagulla, V., Mudapaka, V., & Konka, U. (2025). AI-Driven DevOps in Telecommunications.
11. Herrera, J. L., Montebugnoli, S., Scotece, D., Foschini, L., & Bellavista, P. (2026). A Tutorial on O-RAN Deployment Solutions for 5G: From Simulation to Emulated and Real Testbeds. *IEEE COMST*.
12. Sufyan, A., Khan, K. B., Khashan, O. A., Mir, T., & Mir, U. (2023). From 5G to Beyond 5G: A Comprehensive Survey of Wireless Network Evolution, Challenges, and Promising Technologies.
13. Narasimhulu, M., Mounika, D. V., Varshini, P., Amarendra, K., & Rao, T. K. R. K. (2023). Investigating the Impact of Containerization on the Deployment Process in DevOps.
14. Sakinala, K. (2025). The Synergistic Impact of Artificial Intelligence on DevOps: A Comprehensive Review.
15. Garcia, F. J., et al. (2025). 5G-CT: Automated Deployment and Over-the-Air Testing of End-to-End Open Radio Access Networks. *IEEE*.

THANK YOU

Questions and Suggestions

Team 23UG005 • Panel 4
Project Guide: Ms. Pooja Gowda